

PROYECTO		COMPONENTE		TECNOLOGÍAS		
Donatón	— Sistema	Backend	(Microservicios	Spring	Data	JPA,
Humanitario		Spring Boot)		Hibernate,	PostgreSQL	

1. Arquitectura de Persistencia de Datos

El sistema backend de **Donatón** implementa una arquitectura basada en microservicios, donde cada servicio gestiona su lógica de negocio de manera autónoma. Para asegurar la integridad y la portabilidad del modelo de datos de la plataforma, la capa de persistencia se construye utilizando el estándar industrial **Java Persistence API (JPA)** con **Hibernate** como motor de mapeo objeto-relacional (ORM), conectado a bases de datos relacionales **PostgreSQL**.

Los microservicios involucrados en el acceso a datos son:

- `ms-auth` : Gestión de credenciales y tokens de acceso.
- `ms-donacion` : Registro, estados y flujo de donaciones.
- `ms-logistica` : Centros de acopio, despachos y asignación de recursos.
- `ms-necesidad` : Publicación y seguimiento de necesidades humanitarias.
- `ms-usuarios` : Información detallada de perfiles de donantes, organizaciones y administradores.
- `ms-matching` : Vinculación inteligente entre necesidades y donaciones.
- `ms-notificaciones` : Historial de alertas y preferencias de usuario.

Patrón Shared Kernel (Módulo ms-common)

Para evitar la duplicación de código y garantizar que todos los microservicios manejen definiciones exactas de las entidades y relaciones, se implementó el patrón **Shared Kernel** a través del módulo `ms-common` . Este subproyecto Maven se compila como una librería

compartida y contiene todas las declaraciones de entidades (`@Entity`), repositorios (`JpaRepository`), enums y DTOs comunes.

2. Modelado de Entidades JPA

Las entidades de Donatón se mapean directamente a tablas relacionales de PostgreSQL utilizando anotaciones estándar de Jakarta Persistence y Lombok para reducir el código boilerplate. A continuación, se detalla la estructura y configuración de persistencia del modelo a través del ejemplo de la entidad `Donacion` :

```
package com.donaton.backend.donacion.model;

import jakarta.persistence.*;
import lombok.*;
import java.time.LocalDateTime;
import com.donaton.backend.auth.model.Usuario;
import com.donaton.backend.logistica.model.CentroAcopio;

@Entity
@Table(name = "donaciones")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Donacion {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    @JoinColumn(name = "donante_id", nullable = false)
    private Usuario donante;

    @ManyToOne
    @JoinColumn(name = "centro_acopio_id")
    private CentroAcopio centroAcopio;

    @Column(nullable = false)
    private String descripcion;

    private String categoria;
    private String tipoDonacion;
    private String unidad;
    private String zona;

    @Column(name = "necesidad_id")
    private Long necesidadId;

    private Integer cantidad;

    @Enumerated(EnumType.STRING)
    private EstadoDonacion estado;
```

```
@Column(name = "fecha_creacion")
private LocalDateTime fechaCreacion;

@PrePersist
protected void onCreate() {
    fechaCreacion = LocalDateTime.now();
}

public enum EstadoDonacion {
    PENDIENTE, EN_PROCESO, ENTREGADA, CANCELADA
}
}
```

Detalle del Mapeo Implementado

- **Identificadores Autoincrementales:** Se utiliza la estrategia `GenerationType.IDENTITY` para delegar la creación de llaves primarias en la columna autoincrementable serial de PostgreSQL.
- **Relaciones de Base de Datos:** Se modela la relación `@ManyToOne` para asociar múltiples registros de donaciones a un único registro de usuario (donante) o centro de acopio, generando automáticamente llaves foráneas consistentes.
- **Enums mapeados a Cadenas:** Para asegurar la legibilidad directa de los datos en la base de datos PostgreSQL, los enums como `EstadoDonacion` se guardan en formato de texto mediante la anotación `@Enumerated(EnumType.STRING)`.
- **Eventos de Ciclo de Vida:** Se implementa la anotación `@PrePersist` mediante el método `onCreate()`, el cual inyecta la marca de tiempo de creación local en el registro inmediatamente antes de insertarse en la base de datos.

3. Capa de Acceso a Datos: Patrón Repository

El acceso a la base de datos se desacopla completamente de la lógica del servicio mediante el uso del patrón **Repository** de Spring Data JPA. Al extender de la interfaz `JpaRepository<T, ID>`, el framework inyecta dinámicamente operaciones CRUD estandarizadas (como `save()`, `findById()`, `findAll()` y `deleteById()`) y provee optimizaciones como paginación y ordenamiento automáticos.

Adicionalmente, se configuran métodos de consulta dinámicos basados en la nomenclatura semántica del framework:

```
package com.donaton.backend.donacion.repository;

import com.donaton.backend.donacion.model.Donacion;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface DonacionRepository extends JpaRepository<Donacion, Long> {
    List<Donacion> findByDonanteld(Long donanteId);
    List<Donacion> findByCentroAcopioId(Long centroAcopioId);
    List<Donacion> findByEstado(Donacion.EstadoDonacion estado);
}
```

4. Integración de Procedimientos Almacenados y Triggers

Aunque el estándar arquitectónico de Donatón delega la lógica transaccional de negocio en la capa de servicios de Spring Boot para asegurar la portabilidad e independencia de los microservicios, el sistema está preparado para integrar operaciones intensivas en la base de datos mediante **Procedimientos Almacenados** (Stored Procedures) y **Triggers** nativos de PostgreSQL.

4.1 Mapeo de Procedimientos Almacenados mediante @Procedure

Spring Data JPA provee la anotación `@Procedure` para invocar procedimientos almacenados de base de datos directamente desde el repositorio, mapeando parámetros de entrada y salida a variables Java de manera segura y controlada.

Por ejemplo, supóngase un procedimiento almacenado en PostgreSQL diseñado para procesar y asignar lotes de donaciones acumuladas de forma masiva:

```
-- Definición del Stored Procedure en PostgreSQL
CREATE OR REPLACE PROCEDURE registrar_donacion_sp(
    IN p_donante_id BIGINT,
    IN p_descripcion VARCHAR,
    IN p_cantidad INTEGER,
    OUT p_exito BOOLEAN
)
LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO donaciones (donante_id, descripcion, cantidad, estado, fecha_creacion)
    VALUES (p_donante_id, p_descripcion, p_cantidad, 'PENDIENTE', NOW());
    p_exito := TRUE;
EXCEPTION WHEN OTHERS THEN
    p_exito := FALSE;
END;
$$;
```

Este procedimiento se puede mapear directamente en la interfaz del repositorio Java de la siguiente manera:

```
public interface DonacionRepository extends JpaRepository<Donacion, Long> {

    @Procedure(procedureName = "registrar_donacion_sp")
    Boolean registrarDonacionSp(
        @Param("p_donante_id") Long donanteId,
        @Param("p_descripcion") String descripcion,
        @Param("p_cantidad") Integer cantidad
    );
}
```

4.2 Ejecución de Consultas Nativas y Funciones Complejas

Para consultas altamente específicas que requieren operaciones complejas, joins avanzados o llamados de funciones propias de PostgreSQL que no son fácilmente expresables en JPQL, la plataforma utiliza la anotación `@Query` configurando el parámetro `nativeQuery = true`.

Un ejemplo aplicable al microservicio de logística para calcular la distancia o el balance óptimo entre centros de acopio y donaciones registradas:

```
public interface CentroAcopioRepository extends JpaRepository<CentroAcopio, Long> {

    @Query(value = "SELECT c.*, calcular_distancia(c.latitud, c.longitud, :lat, :lon) AS distancia " +
        "FROM centros_acopio c WHERE c.activo = true " +
        "ORDER BY distancia LIMIT 1",
        nativeQuery = true)
    CentroAcopio buscarCentroMasCercano(
        @Param("lat") Double latitud,
        @Param("lon") Double longitud
    );
}
```

5. Pool de Conexiones y Rendimiento

Para asegurar un excelente tiempo de respuesta bajo alta concurrencia, la persistencia utiliza el pool de conexiones **HikariCP**, integrado de manera predeterminada en Spring Boot. La configuración típica en los archivos `application.properties` de los microservicios encapsula la inyección de credenciales y variables de entorno definidas en contenedores Docker y AWS:

```
# Configuración típica de base de datos relacional (ms-usuarios)
spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USERNAME:postgres}
spring.datasource.password=${DB_PASSWORD:postgres}
spring.datasource.driver-class-name=org.postgresql.Driver

# Configuración del Pool HikariCP
spring.datasource.hikari.maximum-pool-size=10
spring.datasource.hikari.minimum-idle=5
spring.datasource.hikari.idle-timeout=300000
spring.datasource.hikari.connection-timeout=20000

# Parámetros adicionales de JPA / Hibernate
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
```

```
spring.jpa.hibernate.ddl-auto=update  
spring.jpa.show-sql=false
```

Resumen de Parámetros Clave

Parámetro	Propósito	Valor Predeterminado
<code>maximum-pool-size</code>	Define el límite máximo de conexiones activas simultáneas a la base de datos PostgreSQL.	10 conexiones
<code>connection-timeout</code>	Tiempo límite de espera en milisegundos para obtener una conexión del pool antes de arrojar un error.	20,000 ms (20s)
<code>ddl-auto</code>	Estrategia de generación y actualización del esquema de base de datos a partir de las anotaciones JPA.	<code>update</code> (desarrollo) / <code>none</code> (producción)